CS 2110 Homework 5
Intro to C

Elizabeth Hong, Henry Bui, Rohit Rao, Isaac Kim, Owen Anderson,
Bianca Jayaraman, Andy Garcha, Kepler Boyce, Jordan Miao

Spring 2025

Contents

1 Overview **2**
- 1.1 Purpose . . . 2
- 1.2 Task . . . 2
- 1.3 TA Tips . . . 3
- 1.4 Criteria . . . 3

2 Detailed Instructions **4**
- 2.1 `my_string.c` functions . . . 4
- 2.2 `minecraft.c` . . . 7
  - 2.2.1 .h files . . . 7
  - 2.2.2 Functions to Implement in `minecraft.c` . . . 9

3 Deliverables **12**

4 Useful Tips **13**
- 4.1 Man Pages . . . 13
- 4.2 Debugging with GDB and printf . . . 13

5 Checking Your Solution **13**
- 5.1 Makefiles . . . 13
- 5.2 Autograder . . . 13
- 5.3 Manual Testing . . . 14

6 Rules and Regulations **16**
- 6.1 Academic Misconduct . . . 16

1 Overview

1.1 Purpose

The purpose of this assignment is to introduce you to basic C programming along with the tools you need to succeed as a C programmer. This assignment will familiarize you with C syntax and how to compile, run, and debug C. Furthermore, you will gain exposure to common practices such as man(ual) pages and standard libraries which are utilized in real world C programming.

You will become familiar with how to work with strings, arrays, pointers, and structs in C. You will understand the relationship in C between arrays and pointers, including pointer arithmetic (think about how arrays are stored in memory in assembly). You will also become familiar with how to use a Makefile to automate the compilation of your program.

1.2 Task

You will write your C code in two files: `my_string.c`, `minecraft.c`.

See the Detailed Instructions section for more details on the specific requirements for each function.

IMPORTANT NOTE:

- **You need to complete 'string.c' before 'minecraft.c' as you will be using the implemented functions from 'string.c' in 'minecraft.c'.**

In `my_string.c`, you will write your own implementations of common C library functions for working with strings:

- `my_strlen`
- `my_strncmp`
- `my_strncpy`
- `my_strncat`
- `my_memset`
- `reverse_string`
- `is_numeric`

In `minecraft.c`, you will play Minecraft! Your Minecraft world will have the following functions:

- `create_player`
- `delete_player`
- `add_item`
- `spawn_monster`
- `fight_monster`
- `fight_player`
- `fight_ender_dragon`

Take a look at the sections on Makefiles and Autograder for more info on how to compile and test your program.

1.3 TA Tips

While doing the homework, you may find it helpful to draw diagrams of memory locations, as you did with assembly programming. How are arrays represented in memory? How can you use pointers to find the address of `array[i]`? How are arguments passed to a function and results returned using the stack frame? Remember, C functions are pass by value (copies of the values of arguments are pushed onto the stack), just like subroutines in assembly.

1.4 Criteria

Your C code must compile without errors or warnings, using the provided Makefile. Your array of structs should be populated correctly at the end of the program. Your helper functions in `my_string.c` must all be implemented correctly, producing the same behavior for test cases as the equivalent library functions from `string.h`.

2 Detailed Instructions

2.1 `my_string.c` functions

The first part of this homework is to implement a few very common C string library functions found in `string.h`, as well as a couple unique string functions. The caveat is that you must implement these functions **using only pointer notation**. That is, you cannot use array indexing notation, such as `str[i] = 'a'`. This restriction only applies in the `my_string.c` file. We recommend implementing these functions first so you are able to use these functions as you move on with the assignment. Make sure to read all the information in this section to ensure you have all the information you need to succeed!

For this file, you may assume that all inputs are valid (i.e. not NULL).

- `size_t my_strlen(const char *s)`:

  This function calculates the length of the string pointed to by `s`, up to and excluding the null terminator (`'\0'`). Consider the following examples:

  - If `char *pet = "otter"`, then `my_strlen(pet) == 5`. Remember, the null terminator is automatically added here.
  - If `char letters[] = {'a', 'b', 'c', '\0', 'd'}`, then `my_strlen(letters) == 3`.
  - If a string with no null terminator is passed into `my_strlen`, the string standard library defines the output to be undefined behavior. *Consider why this might be the case?*

    Therefore, this function should do nothing extra to handle inputs of invalid strings (i.e., unterminated strings) because the programmer is expected to pass in valid strings.

- `int my_strncmp(const char *s1, const char *s2, size_t n)`:

  This function compares two strings, checking a maximum of `n` characters of both strings. Note that:

  - Comparisons should be made between each character between the ASCII values of each pair of corresponding characters.
  - Comparisons should be character by character until one of the following occurs:
    * There is a difference between the corresponding pair of characters. (This includes if one of the strings terminates before the other one!)
    * Both strings terminate at the same character.
    * We have completed `n` comparisons.

  This function should return an arbitrary negative integer if the first string is lexicographically less than the second string, zero if equal, and positive if greater. You should also remember to account for null terminators. Some examples of string comparisons are below:

  - `my_strncmp("bazz", "bog", 3) < 0`
  - `my_strncmp("rainforest","rain", 4) == 0`
  - `my_strncmp("rainforest","rain", 5) > 0` (recall, `'f' > '\0'`)
  - `my_strncmp("a\0a", "a\0b", 3) == 0` (notice how and where an additional null terminator is placed mid-string!)

- `char *my_strncpy(char *dest, const char *src, size_t n)`:

  This function copies at most `n` characters from a source string (`src`) into a destination memory location (`dest`), returning a pointer to `dest`. Note that:

  - If `src` ends before `n` characters have been copied to `dest`, then continue writing null characters to `dest` until `n` characters have been written.

- If `src` has not ended after `n` characters have been copied to `dest`, a null terminator should not be added to `dest`.
- As an implementer, you can assume `dest` is sufficiently large enough for `n` characters to be written into it (note that if you *use* this function, you have to ensure the `dest` argument you enter is sufficiently large enough!).

Consider the following example. Let us define the following variables:

```
char src[]  = "hob";
char dest[] = "ribbit";
```

Then,

- `my_strncpy(dest, src, 3)` would set dest to {'h', 'o', 'b', 'b', 'i', 't', '\0'}.
- `my_strncpy(dest, src, 4)` would set dest to {'h', 'o', 'b', '\0', 'i', 't', '\0'}.
- `my_strncpy(dest, src, 5)` would set dest to {'h', 'o', 'b', '\0', '\0', 't', '\0'}.

- `char *my_strncat(char *dest, const char *src, size_t n)`:

  This function appends at most `n` bytes from the `src` string to the `dest` string, returning a pointer to `dest`. This function starts writing characters over `dest`'s null terminator and stops writing to `dest` once:

  - a null terminator is found in `src`, or
  - `n` characters have been written from `src`. In this case, a null terminator is added to the end of `dest`.

  Note that, like `my_strncpy`, as the implementor of this function, you can assume `dest` is sufficiently large enough for the result. (If you use this function, you must provide a sufficiently large enough `dest` to hold the result of this function!)

  Consider the following example. Let us define the following variables:

  ```
  char src[6]    = "defgh";
  char dest[100] = "abc";
  ```

  Then (assuming we set `src` and `dest` back to these inputs after every execution),

  - `my_strncat(dest, src, 3)` sets dest to {'a', 'b', 'c', 'd', 'e', 'f', '\0' }
  - `my_strncat(dest, src, 5)` sets dest to {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', '\0' }
  - `my_strncat(dest, src, 6)` also sets dest to {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', '\0' }
  - `my_strncat(dest, src, 9)` also sets dest to {'a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', '\0' }

- `void *my_memset(void *str, int c, size_t n)`:

  This function fills the first `n` bytes of the memory area pointed to by `str` with the constant int value `c`. The function returns a pointer to the memory location `str`.

  Note that this function does not copy the int value `c` directly into `str`, but rather, truncates it to a byte before copying it.

  As an implementer, you can assume the `str` buffer has enough space to write `n` bytes into (If you use this function, you must ensure you provide a `str` buffer that has enough space to write `n` bytes into). This function does not add a null terminator at the end.

  Consider this example. Suppose you had a destination buffer `char str[] = "i love aardvarks!"`:

- `my_memset(str, 'a', 1)` updates `str` to `"a love aardvarks!"`
- `my_memset(str, 'a', 7)` updates `str` to `"aaaaaaaaardvarks!"`
- `my_memset(str, 0x9961, 16)` updates `str` to `"aaaaaaaaaaaaaaaa!"`. Note here that `0x9961` truncates to `0x61` which encodes the character `'a'`.

- `void reverse_string(char *str)`:

  This function reverses a string in place, effectively flipping the order of all characters in the input string. It modifies the input string directly and does not use additional memory beyond temporary integer variables.

  Consider these examples:

  - if `char foo[] = "mississippi"`, then calling `reverse_string(foo)` should update `foo` to `"ippississim"`.
  - if `char bar[] = "bookkeeper"`, then calling `reverse_string(bar)` should update `bar` to `"repeekkoob"`.
  - if `char baz[] = "Zzyzx"`, then calling `reverse_string(bar)` should update `baz` as `"xzyzZ"`.

- `int is_numeric(const char *str)`:

  This function checks whether a given string consists only of numeric digits (`0-9`). It returns `1` if the string is numeric and `0` otherwise.

  For example,

  - `is_numeric("12345")` returns 1, since all characters in the string are numeric characters.
  - `is_numeric("12as")` returns 0, since `a` and `s` are not numeric characters.

You will notice that many of the arguments in the `my_string.c` and `minecraft.c` files are of type `const char *`. This is a pointer to a char that is constant. This means that you cannot change any of the characters to which a `const char *` points. If you attempt to do so, using something like `*pointer = 'c'`, you will get a compile error. If `const` does not precede a `char *` declaration, you can edit the characters to which it points to.

In order to understand the functionalities of these library functions, you may also take a look at their man page (i.e. manual page). See the section on Man Pages for more info.

What is `size_t`:

`size_t` is an unsigned integer datatype in C typically used to represent the size or length of data. In particular, it is large enough to store the result of `sizeof(...)`. The `size_t` type is used throughout the C standard library (which, of course, includes the string library functions).

Some notes:

1. You should complete `my_string.c` before moving on to `minecraft.c`.
2. **You are not allowed to use array notation in this file (e.g. s1[0]). All functions should be implemented using pointers and pointer arithmetic only!** Think about how arrays and pointers correlate with each other in C. Again, this restriction only applies to this file. If you do use array notation in your code, the makefile will indicate this error:
   `make: *** [Makefile:30: check-array-notation] Error 1` along with the instances of array notation that it encountered.
3. You are not allowed to use any of the standard C string libraries (e.g. `#include <string.h>`).
4. Your string functions should not assume the length of any arguments passed in.
5. For `my_strncmp`, you do not need to return a specific number "less than" or "greater than," as long as it follows the description in the `strncmp` man page.

2.2 minecraft.c

The second part of this homework is to implement several C functions within the `minecraft.c` file.

You are feeling nostalgic and have decided you want to feel the thrill of surviving in a Minecraft world. Unfortunately, your poorly performing computer can't run Minecraft (because you don't know how to install Sodium) and you have unchecked hatred for the Java programming language, so you've decided you instead are going to implement Minecraft in C! You have a nagging feeling that you prototyped Minecraft incorrectly, but you've decided that you are just going to roll with it and pretend that your Minecraft in C project **definitely, absolutely, and unequivocably** is a perfect copy of the original behavior of Minecraft. In order to achieve your dreams of writing Minecraft in C, implement the functions found within `minecraft.h` in `minecraft.c`.

The overall gist is as follows: There exists a single world. This world has Players, Monsters, and Items. Players can fight Monsters and other Players using Items. For Player vs. normal Monster altercations, the winner is determined by the Player's level, the Monster's level, and how many of a certain resource a Player has in their Inventory. For Player vs. Player battles, the winner is determined by which Player has a higher leveled item and in the case of a tie, by which Player has higher health. If both are the same, the challenger loses. A Player can also choose to challenge the Ender Dragon. A Player will beat the Ender Dragon if they have a level equal to or higher than 100 and if their Inventory's Item levels sum to greater than or equal to 1500.

Note: For these functions you will not be allowed to import the C library's `string.h`. You will be using your own string functions that you implemented in `my_string.c` for `minecraft.c`. This means if you haven't implemented `my_string.c` correctly, it may affect your `minecraft.c` file as well.

2.2.1 .h files

Before we outline the functions, you should familiarize yourself with C header files. A header file is a C file (by convention with the extension .h) that contains function prototypes, struct definitions, as well as macros. Header files are useful so that we can separate these declarations and definitions from our main C code and later include them in other files. You can see in the code that `minecraft.c` includes `minecraft.h`, its header file.

Here is some of what's defined in `minecraft.h`:

Your `struct World` is defined as such in `minecraft.h`:

```
struct World {
    struct Player players[MAX_PLAYERS];
    int num_players;
    struct Monster monsters[MAX_MONSTERS];
    int num_monsters;
};
```

Your world keeps track of the following information:

- `players`: An array which holds all the players currently in the world,
- `num_players`: The number of players currently in the world,
- `monsters`: An array which holds all of the monsters currently in the world,
- `num_monsters`: The number of how many monsters are currently in the world

The players of your world are represented by the following struct:

```
struct Player {
    char name[MAX_NAME_LENGTH];
    struct Item inventory[MAX_INVENTORY_SIZE];
    int inventory_size;
    int level;
    int health;
};
```

Each `struct Player` tracks the following information:

- `name`: The name of the player,
- `inventory`: An array of the Items the player has,
- `inventory_size`: The number of items the player has in their inventory,
- `level`: The level of the player,
- `health`: The remaining health of the player.

Each Item is represented by the following struct:

```
struct Item {
    char weapon[12];
    int level;
};
```

Each Item (`struct Item`) tracks the following information:

- `weapon`: A string for the name of the item. This value *must* be "Sword", "Bow", or "Ender Pearl",
- `level`: The level of the item.

Each Monster is represented by the following struct:

```
struct Monster {
    char mob[9];
    int level;
};
```

Each Monster (`struct Monster`) tracks the following information:

- `mob`: A string for the type of mob. This value *must* be "Enderman", "Zombie", or "Skeleton",
- `level`: The level of the monster.

Useful Macro Definitions in `minecraft.h`:

- `SUCCESS` is an macro for the integer value to return when a function operation succeeds.
- `FAILURE` is an macro for the integer value to return when a function operation fails. Think of this as an error code.
- `INCOMPLETE` is used as the default return value if you have not completed a function.

- `MAX_NAME_LENGTH` represents the maximum length of the name/mob `char` arrays, including the null terminator. Remember char arrays are one way to represent strings in C.
- `MAX_INVENTORY_SIZE` represents the maximum length of the `inventory` array.
- `MAX_PLAYERS` represents the maximum number of players in the `players` array.
- `MAX_MONSTERS` represents the maximum number of monsters in the `monsters` array.
- `PLAYER_INITIAL_HEALTH` represents the initial starting health of a player.
- `ENDER_DRAGON_HEALTH` represents the health/number of points needed to beat the Ender Dragon.
- `UNUSED_PARAM(x)` is a macro that is used in `minecraft.c` as a placeholder so that you can compile the file without needing to complete every function. You may remove these once you've completed a function.

2.2.2 Functions to Implement in `minecraft.c`

- `int create_player(const char *name)`:

  You need to allow players to join your world. Implement a function that adds a player to the player array in your world.

  - The players in the world's player array should start at index 0 and remain contiguous before and after the operation.
  - Since this player is new to the world, they should have a level of 1, an empty inventory, and `PLAYER_INITIAL_HEALTH` health.
  - If the name is too long or is a null pointer, do not add the new player and return `FAILURE`.
  - If the world is full (i.e., if there are `MAX_PLAYERS` players in the world), do not add the new player and return `FAILURE`.
  - If the world already has a player with the name passed-in to the function, do not add the new player and return `FAILURE`.
  - Otherwise, add the player to the world, update any attributes necessary, and return `SUCCESS`.

- `int delete_player(const char *player_name)`:

  If a player dies (health is less than or equal to 0) or logs off, then they should be deleted from the world. Implement a method to delete a player.

  - If the player name passed in is `NULL`, return `FAILURE`.
  - The world's players array should start at index 0 and remain contiguous before and after the operation.
  - If no player in the world's player array has the specified name, do not remove any player and return `FAILURE`.
  - Otherwise, remove the player from the players array and return `SUCCESS`.

- `int add_item(struct Player *player, char *weapon, int level)`:

  You need a way to add items to a player's inventory. `add_item` adds a given item to a player's inventory. **Note:** The inventory array is structured such that each individual item takes up a space; items do not stack. That means an Inventory array for a player with 2 bows would look like (at a high level)
  `[{weapon: "Bow", level: 12},{weapon: "Bow", level: 5}]`
  rather than having one Bow item with an amount attribute of 2.

  - If the player passed in is `NULL`, return `FAILURE`

- If the player's inventory is full (i.e., they have `MAX_INVENTORY_SIZE Items` in their inventory), return `FAILURE`,
- If the name passed in is `NULL` or not "Bow", "Ender Pearl", or "Sword", return `FAILURE`.
- If the level passed in is less than 1, return `FAILURE`
- Otherwise, create an Item and put it in the first available spot in the player's inventory array. Make sure to update the number of items the player has as well.
- If the item is successfully added, return `SUCCESS`.

- `int spawn_monster(const char *mob, int level)`:

  The players in your world are lonely. You need to allow monsters to spawn in your world. Implement a function that creates a mob of the passed in `mob` and adds it to the monsters array in your world.

  - The monsters in the world's monster array should remain contiguous before and after the operation and should start at index 0.
  - If the name passed in is not "Enderman", "Zombie", or "Skeleton", do not add the monster and return `FAILURE`.
  - If the inputted level is less than 1, do not add any monster and return `FAILURE`.
  - If the world's monster array is full (i.e., if there are `MAX_MONSTERS` monsters in the world), do not add any monster and return `FAILURE`.
  - Otherwise, add the monster and return `SUCCESS`.

- `int fight_monster(struct Player *player)`:

  You need an outlet for your rage after dropping all your diamonds in a pit of lava. Implement a function that allows a player to fight one of the monsters in the world.

  - The player should fight the **last monster** in the world's monster array.
  - If the player passed in is `NULL`, return `FAILURE`.
  - If there are no monsters in the world, return `FAILURE`.
  - The outcome of the fight should be determined by how many of the monster's weakness the player has in their inventory scaled by the player's level. The weaknesses are as follows: Skeletons are weak to Ender Pearls, Zombies are weak to Bows, and Endermen are weak to Swords.
  - A player wins the battle if `(# of weakness items) * (player's level) > (monster's level)`. For example, if a player is fighting a level 7 Skeleton, the player would win if they were level 8 and had 1 Ender Pearl *or* if they were level 1 and had 8 Ender Pearls.
  - If the player wins, the monster is removed from the world and the player gets a monster's drop item at the monster's level. The drop items are as follows: Skeletons drop Bows, Endermen drop Ender Pearls, and Zombies drop Swords. In the above example, the player would receive a level 8 Bow for defeating a level 8 Skeleton. The player's level should also be increased by 1.
  - If the player fails to win (`(# of weakness items) * (player's level) <= (monster's level)`), the player loses health equal to the monster's level. If a player with 20 health loses to a level 15 Enderman, they would have 5 health remaining. Both the player and the Enderman remain in the world, and the Enderman would remain unchanged.
  - If a player loses and their health falls equal to or below 0, delete the player.
  - Return 0 if the player wins and 1 if the monster wins.

Monster	Zombie	Skeleton	Enderman
Weakness	Bow	Ender Pearl	Sword
Drop Item	Sword	Bow	Ender Pearl

- `int fight_player(struct Player *challenger, struct Player *opponent)`:

  Your friend stole your pink sheep again and you've just about had it up to here. Implement the `fight_player` function so they know not to make that mistake again.

  - If any parameter is `NULL`, return `FAILURE`.
  - If either player has an empty inventory, return `FAILURE`
  - The winner of the fight is determined by who has the item with the highest level.
  - If both `challenger` and `opponent` have the same maximum level of item, then the tie should be broken by whichever player has a higher health. If this also ties, then the challenger should win.
  - The loser of the fight should have their health decreased by the *level of* the highest level item of the winner. In other words, the loser should lose health equal to the level of the highest level item found when looking for the winner.
  - After the fight has ended:
    - Return `0` if the challenger wins
    - Return `1` if the opponent wins
  - For example, if Player 1's highest level item is a level 9 Bow and Player 2's highest level item is a level 11 Sword, Player 2 wins and Player 1's health is decremented by 11. If both Player 1 and Player 2's highest item's were level 11, the Player with higher health would win and the losing player's health would be decremented by 11.
  - If a player's health falls equal to or below 0, delete the player.
  - The winning player does not receive anything beyond the gratification of vanquishing their enemy.

- `int fight_ender_dragon(struct Player *player)`:

  In case a player becomes bored of normal mobs and players (we've intentionally omitted Creepers from this homework to accelerate such boredom as they are the most melodramatic mob), they should be able to fight the Ender Dragon. Implement a method to simulate the outcome of a player's fight with the Ender Dragon.

  - If the player passed in is `NULL`, return FAILURE.
  - The player must be at least level 100 to challenge the ender dragon. If the player is unable to challenge the dragon, return `FAILURE`.
  - The outcome of the fight should be determined by the sum of the levels of items in the player's inventory. For example, if a player has 20 items each of level 80, their total sum would be $20 * 80 = 1600$. If the player's sum is greater than `ENDER_DRAGON_HEALTH`, the player wins. Otherwise, the player loses.
  - If the player wins the battle:
    - Increase the player's level by 25.
    - Restore the player's health to full.
    - Return `SUCCESS`.
  - If the dragon wins the fight, delete the player and return `FAILURE`.

Hints:
In regards to the functions to be implemented, here are some common mistakes and reminders for how to go about solving them.

- `MAX_NAME_LENGTH` represents the maximum lengths of the name `char` array. However, length in this case doesn't necessarily mean the length from `my_strlen`, which by definition, does *not* include the null terminator.

- Remember you can index into strings for reading or writing characters. For example, `some_string[3]` is the 4th character of `some_string`. **In this file, you are allowed to use array notation.**
- When using any of the functions that write to a destination buffer (e.g., `my_strncpy`, `my_strncat`, `my_memset`), ensure that your `dest` buffer has enough capacity for that function. As examples,
  - For `my_strncpy` and `my_memset`, `n` should never exceed `sizeof(dest)` (supposing that `dest` is a char array).
  - For `my_strncat`, `n` should never exceed `sizeof(dest) - strlen(dest)` (also supposing that `dest` is a char array).
- When comparing strings to each other, be wary of the `n` argument you pass in to `my_strncmp` that tells how many characters you will be comparing.
- Remember all strings are to be null-terminated! Otherwise, we would not be able to tell what characters are actually part of a string.
- **Always check the validity of your parameters.** Possible things to check for:
  - `NULL` values
  - Strings that are too long
  - Values that are outside the acceptable range

  If you have an invalid input, the function should not do anything, and return `FAILURE`.

3 Deliverables

Please upload the following files to Gradescope:

1. `my_string.c`
2. `minecraft.c`

Note: Please do not wait until the last minute to run/test your homework; history has proven that last minute turn-ins will result in long queue times for grading on Gradescope.

4 Useful Tips

4.1 Man Pages

The `man` command in Linux provides "an interface to the on-line reference manuals." If you are unsure about the specifics of a my_string.c function, you should look up the exact details using its man page. This is a great utility for any C and Linux developer for finding out more information about the available functions and libraries. In order to use this, you just need to pass in the function name to this command within a Linux (in our case Docker) terminal.

For instance, entering the following command will print the corresponding man page for the strlen function:

```
$ man strlen
```

Additionally, the man pages are accessible online at: http://man.he.net

NOTE: You can ignore the subsections after the "RETURN VALUE" (such as ATTRIBUTES, etc) for this homework, however, pay close attention to function descriptions.

4.2 Debugging with GDB and printf

We highly recommend getting used to "`printf` debugging" in C early on.

Moreover, If you run into a problem when working on your homework, you can use the debugging tool, GDB, to debug your code! Former TA Adam Suskin made a series of tutorial videos which you can find here. GDB is an essential tool for any C program, so we definitely recommend learning it now.

When running GDB, if you get to a point where user input is needed, you can supply it just like you normally would. When an error happens, you can get a Java-esque stacktrace using the `backtrace` (`bt`) command which allows you to pinpoint where the error is coming from. For more info on basic GDB commands, search up "GDB Cheat Sheet".

5 Checking Your Solution

5.1 Makefiles

Make is a common build tool for abstracting the complexity of working with compilers directly. Makefiles let you define a set of desired targets (files you want to compile), their prerequisites (files which are needed to compile the target), and sets of directives (commands such as gcc, gdb, etc.) to build those targets. In all of our C assignments (and also in production level C projects), a Makefile is used to compile C programs with a long list of compiler flags that control things like how much to optimize the code, whether to create debugging information for gdb, and what errors we want to show. We have already provided you a Makefile for this homework, but we highly recommend that you take a look at this file and understand the `gcc` commands and flags used to understand how to compile C programs. If you're interested, you can also find more information regarding Makefiles here.

Since your program is connected to an autograder with multiple files that need to be compiled using particular settings, it's a little difficult to compile it by hand. The Makefile allows us to simply type the command `make` followed by a target such as `tests`, `run-tests`, or `run-gdb` to compile and run your code.

5.2 Autograder

The autograder must be run with the provided Docker container. To use the Docker container, open Docker Desktop, `cd` into the project directory and enter:

```
# macOS or Linux
./cs2110docker.sh

# Windows
.\cs2110docker.bat
```

If you are on macOS or Linux, you may get an error such as

```
zsh: permission denied: ./cs2110docker-c.sh
```

If so, make sure to enable execute permissions on the script: `chmod +x ./cs2110docker-c.sh`. You should only need to do this once.

Once you are in the Docker container, you can run the autograder:

```
# Clean your directory (remove any made files)
$ make clean

# Build and run all tests
$ make

# Build and run one test
$ make TEST=test_my_strlen::basic_1

# Build and run all tests in a single file
$ make TEST=test_my_strlen::*

# Build and run a test in GDB
# run-gdb takes the same parameters above
$ make run-gdb TEST=test_my_strlen::basic_1
```

The autograder format will look like the following:

```
Checking for array notation in my_string.c...
No array notation found!
[----] suites/my_strlen.c:14: Assertion Failed
[----]
[----]   Expected length to be correct
[----]
[----]   eq(sz, actual, expected):
[----]     Actual: 1234
[----]     Expected: 5
[FAIL] test_my_strlen::basic_1: (0.00s)
[====] Synthesis: Tested: 1 | Passing: 0 | Failing: 1 | Crashing: 0
```

Here, the top red value (Actual) represents what your solution produced and the bottom green value (Expected) represents the correct value.

Note that the test cases are executed in parallel, so `[FAIL]` headers may not align exactly with the test failure message. If you want to test cases to run one by one, you can run `make test && ./tests -j1`.

5.3 Manual Testing

If you want to write manual tests of your functions, you are allowed to modify `main.c` for use as a driver program. For example, you may want to create a new helper method that will print out the contents of the

trainers array within `minecraft.h`, implement it in `minecraft.c`, and call it in `main.c`. However, if you choose to create extraneous helper methods to help debug **make sure to remove them when submitting to the autograder**. Editing `main.c`, however, should not interfere with the autograder.

Here is how to manually run your `main.c` code.

```
# Clean up all compiled output
$ make clean

# Recompile the driver executable
$ make driver

# Run the driver executable
$ ./driver
```

Important Notes:

1. Only the `.c` file will be graded on Gradescope.
2. All non-compiling solutions will receive a zero (with all the flags specified in the Makefile/Syllabus).
3. All solutions that do not meet syntax restrictions will also receive a zero.
4. **NOTE: DO NOT MODIFY THE HEADER FILES.**

   Since you are not turning them in, any changes to the `.h` files will not be reflected when running the Gradescope autograder.

We reserve the right to update the autograder and the test case weights on Gradescope or the local checker as we see fit when grading your solution.

6 Rules and Regulations

1. Please read the assignment in its entirety before asking questions.
2. Please start assignments early, and ask for help early. Do not email us the night the assignment is due with questions.
3. If you find any problems with the assignment, please report them to the TA team. Announcements will be posted if the assignment changes.
4. You are responsible for turning in assignments on time. This includes allowing for unforeseen circumstances. If you have an emergency please reach out to your instructor and the head TAs **IN ADVANCE** of the due date with documentation (i.e. note from the dean, doctor's note, etc).
5. You are responsible for ensuring that what you turned in is what you meant to turn in. No excuses if you submit the wrong files, what you turn in is what we grade. In addition, your assignment must be turned in via Gradescope. Email submissions will not be accepted.
6. See the syllabus for information regarding late submissions; any penalties associated with unexcused late submissions are non-negotiable.

6.1 Academic Misconduct

Academic misconduct is taken very seriously in this class. Quizzes, timed labs and the final examination are individual work. Homework assignments will be examined using cheat detection programs to find evidence of unauthorized collaboration.

You are expressly forbidden to supply a copy of your homework to another student. If you supply a copy of your homework to another student and they are charged with copying, you will also be charged. This includes storing your code on any platform which would allow other parties to it (public repositories, pastebin, etc). If you would like to use version control, use a private repository on github.gatech.edu

Homework collaboration is limited to high-level collaboration. Each individual programming assignment should be coded by you. You may work with others, but each student should be turning in their own version of the assignment.

High-level collaboration means that you may discuss design points and concepts relevant to the homework with your peers, share algorithms and pseudo-code, as well as help each other debug code. What you shouldn't be doing, however, is pair programming where you collaborate with each other on a single instance of the code, or providing other students any part of your code.

Submissions that are essentially identical will receive a zero and will be sent to the Dean of Students' Office of Academic Integrity. Submissions that are copies that have been superficially modified to conceal that they are copies are also considered unauthorized collaboration.

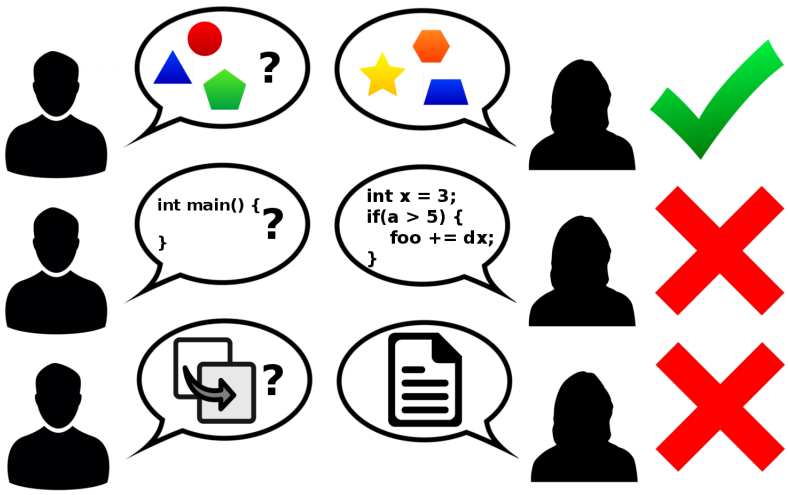

Figure 1: Collaboration rules, explained colorfully